

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 4**



Sorting

Oleh:

Rahma Syarita Jaya NIM. 2510817320004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 4

Laporan Praktikum Algoritma & Struktur Data Modul 4: Sorting ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Rahma Syarita Jaya
NIM : 2510817320004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
SOAL 1: Implementasi Stack	Kesalahan! Bookmark tidak ditentukan.
A. Source Code.....	6
B. Pembahasan	7
SOAL 2: Si Palui dan Mesin Hitung Aneh	Kesalahan! Bookmark tidak ditentukan.
A. Source Code.....	8
B. Output Program	8
C. Pembahasan	9
SOAL 3: Implementasi Queue	10
A. Source Code.....	10
B. Pembahasan	11
SOAL 4: Si Palui dan Deretan Angka	Kesalahan! Bookmark tidak ditentukan.
A. Source Code.....	12
B. Output Program	13
C. Pembahasan	13
TAUTAN GIT	21

DAFTAR GAMBAR

Gambar 1. Output Algoritma Bubble Sort	6
Gambar 2. Visualisasi singkat Bubble Sort.....	7
Gambar 3. Output Algoritma Selection Sort	8
Gambar 4. Output Algoritma Insertion Sort.....	10
Gambar 5. Output Algoritma Merge Sort.....	13
Gambar 6. Output Algoritma Quick Sort	15
Gambar 7. Output Algoritma Radix Sort.....	18
Gambar 8. Gambaran Kumulatif	19

DAFTAR TABEL

Tabel 1 Source code Bubble Sort	6
Tabel 2 Source code Selection Sort.....	8
Tabel 3 Source code Insertion Sort	10
Tabel 4 Source code Merge Sort	12
Tabel 5 Source code Quick Sort	14
Tabel 6 Source code Radix Sort	17

Bubble Sort

A. Source Code

Tabel 1 Source code Bubble Sort

```

1 void bubble_sort(std::vector<int>& data, Metrics& m) {
2     int n = data.size();
3     auto start = Clock::now();
4
5     for (int i=0; i < n-1; i++) {
6         for (int j=0; j < n-i- 1; j++) {
7             m.comparisons++;
8             if (data[j] > data[j + 1]) {
9                 std::swap(data[j], data[j + 1]);
10                m.swaps++;
11            }
12        }
13    }
14    m.time_ms = std::chrono::duration<double,
15    std::milli>(Clock::now() - start).count();
16 }

```

B. Output Program

>> Bubble Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4950	2564	0	0	0	0.016300
Bubble Sort	Sorted	100	4950	0	0	0	0	0.002800
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.023100
Bubble Sort	Nearly Sorted	100	4950	296	0	0	0	0.004600
Bubble Sort	Many Duplicates	100	4950	2315	0	0	0	0.014900
Bubble Sort	Random	1000	499500	243190	0	0	0	1.178100
Bubble Sort	Sorted	1000	499500	0	0	0	0	0.277200
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	2.514700
Bubble Sort	Nearly Sorted	1000	499500	25778	0	0	0	0.357600
Bubble Sort	Many Duplicates	1000	499500	240808	0	0	0	1.408800
Bubble Sort	Random	10000	49995000	25118148	0	0	0	127.826200
Bubble Sort	Sorted	10000	49995000	0	0	0	0	21.484700
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	218.577200
Bubble Sort	Nearly Sorted	10000	49995000	3119352	0	0	0	35.654900
Bubble Sort	Many Duplicates	10000	49995000	25093086	0	0	0	128.146100

Gambar 1. Output Algoritma Bubble Sort

C. Pembahasan

Dalam suatu perulangan pertama sebanyak $n - 1$ dimana n adalah size data tersebut, terdapat lagi perulangan didalamnya untuk membandingkan data pada indeks j atau data pertama yang akan dibandingkan dengan data pada indeks $j+1$ atau data di sebelahnya.

Jika $\text{data}[j]$ lebih besar dari $\text{data}[j+1]$ yang dibandingkan, maka tukar data tersebut. Selebihnya perulangan kedua ini akan terus terulang sampai $n - i - 1$ kali.

Setelah perulangan pertama fase i telah berakhir, data dengan ukuran terbesar sudah berada pada posisi paling belakang.



Gambar 2. Visualisasi singkat Bubble Sort

Dalam ketiga kasus, Worst case, Average case dan Best case. Kompleksitas waktunya adalah $O(n^2)$ pada ketiga kasus tersebut. Setiap jumlah data bertambah, maka jumlah operasi yang dibutuhkan program akan menjadi ganda juga secara eksponensial,

berarti bahwa waktu eksekusi suatu algoritma bertambah secara kuadratik seiring dengan bertambahnya ukuran input.

Algoritma ini termasuk stable karena tetap mempertahankan urutan relatif setelah proses pengurutan selesai. Algoritma ini termasuk algoritma In-Place Sorting karena tidak membutuhkan variabel atau rekursif tambahan.

Selection Sort

A. Source Code

Tabel 2 Source code Selection Sort

```

1 void selection_sort(std::vector<int>& data, Metrics& m){
2     int n = data.size();
3     auto start = Clock::now();
4
5     for(int i = 0; i < n - 1; ++i){
6         int min_idx = i;
7         for(int j = i + 1; j < n; ++j){
8             m.comparisons++;
9             if(data[j]<data[min_idx]){
10                 min_idx = j;
11             }
12         }
13         if (min_idx != i){
14             std::swap(data[i], data[min_idx]);
15             m.swaps++;
16         }
17         m.time_ms = std::chrono::duration<double,
18 std::milli>(Clock::now() - start).count();
19     }
20 }

```

B. Output Program

>> Selection Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.010500
Selection Sort	Sorted	100	4950	0	0	0	0	0.010300
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.010500
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.010400
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.010500
Selection Sort	Random	1000	499500	995	0	0	0	0.897800
Selection Sort	Sorted	1000	499500	0	0	0	0	0.869300
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	0.854600
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	0.869900
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	0.871500
Selection Sort	Random	10000	49995000	9992	0	0	0	83.594000
Selection Sort	Sorted	10000	49995000	0	0	0	0	84.382600
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	83.730600
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	88.024900
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	86.022000

Gambar 3. Output Algoritma Selection Sort

C. Pembahasan

Dalam suatu perulangan pertama sebanyak $n - 1$ dimana n adalah size data tersebut dan membuatnya menjadi indeks nilai minimum sementara $\text{min_idx} = i$.

Di dalamnya, terdapat lagi perulangan kedua yang berjalan sebanyak $n - i - 1$ kali untuk mencari data paling kecil dengan cara membandingkan data tersebut dengan $\text{data}[j]$. Jika $\text{data}[j]$ lebih kecil maka min_idx akan diubah nilainya menjadi j . Dimana ini menyimpan indeks suatu data terkecil.

Setelah perulangan kedua selesai, program akan mengecek apakah min_idx sudah berubah dengan $\text{min_idx} \neq i$. Jika iya, maka dilakukan satu kali penukaran data dengan std::swap . Jika tidak ada perubahan, posisi data dibiarkan. Setelah seluruh i fase perulangan pertama berakhir, data dengan ukuran terkecil akan berada di posisi paling depan.

Dalam kasus Best, Average dan Worst nilai Comparisons konsisiten pada jenis-jenis data yang dimana kompleksitas waktunya adalah $O(n^2)$, waktu eksekusi algoritma ini bertambah secara kuadratik seiring dengan bertambahnya ukuran data.

Algoritma Selection Sort termasuk Unstable karena dapat mengubah urutan relatif data yang memiliki nilai yang sama. Algoritma ini membutuhkan satu alokasi memori tambahan untuk menyimpan variabel indeks sementara.

Insertion Sort

A. Source Code

Tabel 3 Source code Insertion Sort

```

1 void insertion_sort(std::vector<int>& data, Metrics& m)
  {
2     int n = data.size();
3     auto start = Clock::now();
4
5     for (int i = 1; i < n; ++i) {
6         int key = data[i];
7         int j = i - 1;
8         m.comparisons++;
9
10        while (j >= 0 && data[j] > key) {
11            data[j + 1] = data[j];
12            m.shifts++;
13            j = j - 1;
14            m.comparisons++;
15        }
16        data[j + 1] = key;
17    }
18    m.time_ms = std::chrono::duration<double,
19               std::milli>(Clock::now() - start).count();
  }

```

B. Output Program

>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)

Insertion Sort	Random	100	2663	0	2564	0	0	0.002400
Insertion Sort	Sorted	100	99	0	0	0	0	0.000100
Insertion Sort	Reverse Sorted	100	5049	0	4950	0	0	0.006200
Insertion Sort	Nearly Sorted	100	395	0	296	0	0	0.000700
Insertion Sort	Many Duplicates	100	2414	0	2315	0	0	0.002100
Insertion Sort	Random	1000	244189	0	243190	0	0	0.109500
Insertion Sort	Sorted	1000	999	0	0	0	0	0.000700
Insertion Sort	Reverse Sorted	1000	500499	0	499500	0	0	0.240300
Insertion Sort	Nearly Sorted	1000	26777	0	25778	0	0	0.031100
Insertion Sort	Many Duplicates	1000	241807	0	240808	0	0	0.119900
Insertion Sort	Random	10000	25128147	0	25118148	0	0	11.492300
Insertion Sort	Sorted	10000	9999	0	0	0	0	0.005800
Insertion Sort	Reverse Sorted	10000	50004999	0	49995000	0	0	22.017600
Insertion Sort	Nearly Sorted	10000	3129351	0	3119352	0	0	1.395600
Insertion Sort	Many Duplicates	10000	25103085	0	25093086	0	0	10.702500
>> Merge Sort								

Gambar 4. Output Algoritma Insertion Sort

C. Pembahasan

Algoritma ini melakukan perulangan dengan indeks $i = 1$ sampai i tidak melebihi n dimana n adalah size data tersebut. Program akan mengambil elemen pada indeks saat ini menjadi $key = data[i]$ sebagai pembanding nantinya dan $j = i - 1$ sebagai batas indeks.

Terdapat perulangan kedua dengan struktur `while` untuk mengecek data ke kiri selama syarat masih $j \geq 0$ dan nilai $data[j] > key$. Jika kondisi perulangan `while` terpenuhi, maka data indeks j akan digeser satu posisi ke kanan dengan $data[j + 1] = data[j]$. Nilai j akan terus dikurangi satu setiap bergeser. Di luar loop, mengubah key $data[j + 1] = key$ untuk untuk menaruh elemen key ke indeks yang sudah benar.

Setelah seluruh fase i perulangan pertama berakhir, seluruh data sukses terurut dari elemen terkecil di depan hingga terbesar di belakang.

Kompleksitas waktu pada Best Case adalah $O(n)$ karena tidak ada syarat yang memenuhi loop `while` karena sudah terurut. Sedangkan Average dan Worst, nilai Comparisons dan Shifts tinggi dan waktu eksekusi bertambah secara eksponensial seiring bertambahnya ukuran data.

Algoritma ini stabil karena ketika ada data dengan nilai sama, data tersebut akan tetap digeser di sebelah kanan data dengan nilai yang sama yang lama, sehingga urutan relatif asli tidak berubah. Algoritma ini in-place karena hanya memerlukan satu memori tambahan sementara tanpa memerlukan memerlukan panggilan rekursif.

Merge Sort

A. Source Code

Tabel 4 Source code Merge Sort

```
1 void merge(std::vector<int>& data, int left, int mid, int
  right, Metrics& m){
2     int n1 = mid - left + 1;
3     int n2 = right - mid;
4
5     std::vector<int> L(n1), R(n2);
6
7     for (int i = 0; i < n1; i++) L[i] = data[left + i];
8     for (int j = 0; j < n2; j++) R[j] = data[mid + 1 +
  j];
9
10    int i = 0, j = 0, k = left;
11
12    while (i < n1 && j < n2) {
13        m.comparisons++;
14        if (L[i] <= R[j]) {
15            data[k] = L[i];
16            i++;
17        } else {
18            data[k] = R[j];
19            j++;
20        }
21        k++;
22    }
23
24    while (i < n1){ data[k] = L[i]; i++; k++; }
25    while (j < n2){ data[k] = R[j]; j++; k++; }
26 }
27
28 void divide(std::vector<int>& data, int left, int right,
  Metrics& m) {
29     m.recursive_calls++;
30     if (left >= right) return;
31
32     int mid = left + (right - left) / 2;
33     divide(data, left, mid, m);
34     divide(data, mid + 1, right, m);
35     merge(data, left, mid, right, m);
36 }
37
38 void merge_sort(std::vector<int>& data, Metrics& m) {
39     int n = data.size();
40     if (n <= 1) return;
```

```

41
42     auto start = Clock::now();
43     divide(data, 0, n - 1, m);
44     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
45 }

```

B. Output Program

>> Merge Sort									
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)	
Merge Sort	Random	100	535	0	0	199	0	0.012800	
Merge Sort	Sorted	100	356	0	0	199	0	0.019600	
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.012800	
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.009700	
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.017500	
Merge Sort	Random	1000	8690	0	0	1999	0	0.105200	
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.162600	
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	0	0.131300	
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.135800	
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.138300	
Merge Sort	Random	10000	120465	0	0	19999	0	1.578200	
Merge Sort	Sorted	10000	69008	0	0	19999	0	1.615900	
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	1.379700	
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	1.439600	
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	1.706000	

Gambar 5. Output Algoritma Merge Sort

C. Pembahasan

Pada baris [1 sampai 4] prob2.cpp, syntax `#include <iostream>` untuk mengimpor input/output dan memanggil header "queue.h" agar program utama mengenali fungsi yang telah dibuat.

Dalam algoritma Merge Sort, terdapat tambahan 2 rekursif di source code. Masing-masing kegunaannya sebagai berikut:

Di dalam fungsi merge, membuat dua buah vektor sementara yaitu L dan R untuk menyalin data kiri dan kanan. Terdapat perulangan `while i < n1 && j < n2` untuk membandingkan data dua vektor tersebut. Jika `L[i] <= R[j]`, maka data dari vektor kiri akan dimasukkan ke vektor utama `data[k] = L[i]`, jika tidak maka data vektor kanan yang akan dimasukkan ke `data[k] = R[j]`. Sisa-sisa data yang belum dimasukkan akan dilanjutkan oleh dua loop `while` selanjutnya. Setelah seluruh fase perulangan rekursif merge berakhir, seluruh data akan terurut dari yang terkecil.

Dalam fungsi divide yang menerima nilai left dan right untuk membelah data menjadi dua bagian berdasarkan nilai tengahnya `mid = left + (right - left) / 2`. Fungsi ini memanggil dirinya sendiri `divide(data, left, mid, m)` untuk membelah diri lagi bagian kiri dan `divide(data, mid + 1, right, m)` untuk bagian kanan. Perulangan rekursif ini akan terus berjalan dan berhenti ketika `left >= right` yang berarti data sudah terurut sampai tersisa satu elemen. Setelah itu, program memanggil fungsi merge untuk menggabungkan kembali potongan-potongan atau array dari data tersebut.

Fungsi `merge_sort` sendiri hanya mempersiapkan batas indeks data, 0, n-1, m dan mencatat waktu eksekusi. Dalam ketiga kasus, kompleksitas waktunya adalah $O(n \log n)$ karena nilai Comparisons dan Rec.Calls stabil pada jenis-jenis data karena pembelahan data yang dibentuk akan selalu sama alurnya. Algoritma ini stabil karena penggunaan tanda $\leq$, sehingga ketika ada data bernilai sama, nilai relatif asli tidak berubah. Algoritma ini tidak termasuk in-place karena membutuhkan rekursif tambahan.

Quick Sort

A. Source Code

Tabel 5 Source code Quick Sort

1	<code>void partition(std::vector<int>& data, int low, int high, Metrics& m) {</code>
2	<code> m.recursive_calls++;</code>
3	
4	<code> if (low >= high) return;</code>
5	
6	<code> int pivot = data[high];</code>
7	<code> int i = (low - 1);</code>
8	
9	<code> for (int j = low; j <= high - 1; j++){</code>
10	<code> m.comparisons++;</code>
11	<code> if (data[j] <= pivot){</code>
12	<code> i++;</code>
13	<code> std::swap(data[i], data[j]);</code>
14	<code> m.swaps++;</code>
15	<code> }</code>
16	<code> }</code>
17	
18	<code> std::swap(data[i + 1], data[high]);</code>
19	<code> m.swaps++;</code>
20	
21	<code> int pi = i + 1;</code>
22	
23	<code> partition(data, low, pi - 1, m);</code>
24	<code> partition(data, pi + 1, high, m);</code>
25	<code>}</code>
26	
27	<code>void quick_sort(std::vector<int>& data, Metrics& m) {</code>
28	<code> int n = data.size();</code>
29	<code> if (n <= 1) return;</code>
30	
31	<code> auto start = Clock::now();</code>
33	<code> partition(data, 0, n - 1, m);</code>
34	<code> m.time_ms = std::chrono::duration<double,</code>
35	<code>std::milli>(Clock::now() - start).count();</code>
36	<code>}</code>

B. Output Program

```
>> Quick Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	676	396	0	131	0	0.003300
Quick Sort	Sorted	100	4950	5049	0	199	0	0.004000
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.003500
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.003100
Quick Sort	Many Duplicates	100	832	707	0	181	0	0.002200
Quick Sort	Random	1000	10530	6168	0	1337	0	0.033400
Quick Sort	Sorted	1000	499500	500499	0	1999	0	0.345700
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	0.198000
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.045100
Quick Sort	Many Duplicates	1000	12588	9665	0	1801	0	0.037500
Quick Sort	Random	10000	149797	77759	0	13347	0	0.388500
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	25.188100
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	17.405100
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	0.449700
Quick Sort	Many Duplicates	10000	170629	113205	0	18007	0	0.356900

Gambar 6. Output Algoritma Quick Sort

C. Pembahasan

Algoritma ini melakukan perulangan dengan indeks $i = 1$ sampai i tidak melebihi n dimana n adalah size data tersebut. Program akan mengambil elemen pada indeks saat ini menjadi $key = data[i]$ sebagai pembanding nantinya dan $j = i - 1$ sebagai batas indeks.

Di dalam fungsi `partition`, Jika syarat $low \geq high$ terpenuhi, fungsi akan berhenti karena sub-array tidak bisa dibagi lagi. Jika tidak, program akan menentukan data terakhir sebagai pembatas $pivot = data[high]$ dan indeks $i = low - 1$.

Terdapat perulangan kedua `for` dimana melakukan perulangan dari $j = low$ sampai $high - 1$ untuk mengecek data. Di dalam loop, terdapat perbandingan jika $data[j] \leq pivot$, maka $i++$ dan melakukan penukaran posisi menggunakan `std::swap(data[i], data[j])`.

Setelah perulangan selesai, program menempatkan pivot dengan menukarnya via `std::swap(data[i + 1], data[high])`. Langkah terakhir pada fungsi `partition` ini adalah memanggil dirinya dua kali dengan melalui `partition(data, low, pi - 1, m)` untuk memproses sisi kiri pivot, dan `partition(data, pi + 1, high, m)` untuk memproses sisi kanan pivot. Setelah seluruh fase perulangan rekursif berakhir, data akan terurut dari elemen terkecil.

Dalam fungsi utama `quick_sort`, pertama, menyimpan ukuran data menggunakan `int n = data.size()`. Jika data hanya berisi satu elemen atau kosong $n \leq 1$, maka program akan berhenti. Jika data lebih dari satu elemen, memulai waktu awal eksekusi. Lalu, memanggil `partition(data, 0, n - 1, m)` untuk memulai proses pembagian dan pengurutan data indeks paling awal 0 dan paling akhir $n - 1$.

Berdasarkan output performa algoritma Quick Sort ini menunjukkan perbedaan best dan average case dengan worst case. Dimana best dan average case $O(n \log n)$ cenderung stabil berdasarkan waktu esekusiya. Worst Case termasuk $O(n^2)$ karena jumlah operasi perbandingan dan waktu eksekusinya meningkat secara eksponensial.

Algoritma Quick Sort Unstable karena roses pemindahan data bisa melompati data dengan nilai sama sehingga dapat merusak urutan relatif. Algoritma ini termasuk In-Place Sorting karena masih menggunakan `std::swap`.

Radix Sort

A. Source Code

Tabel 6 Source code Radix Sort

1	<code>int getMax(const std::vector<int>& data, int n, Metrics& m) {</code>
2	<code> int mx = data[0];</code>
3	<code> m.array_accesses++;</code>
4	<code> for (int i = 1; i < n; i++) {</code>
5	<code> if (data[i] > mx) {</code>
6	<code> mx = data[i];</code>
7	<code> }</code>
8	<code> }</code>
9	<code> return mx;</code>
10	<code>}</code>
11	
12	<code>void countSort(std::vector<int>& data, int n, int exp, Metrics& m) {</code>
13	<code> std::vector<int> output(n);</code>
14	<code> int count[10] = { 0 };</code>
15	
16	<code> for (int i = 0; i < n; i++) {</code>
17	<code> count[(data[i] / exp) % 10]++;</code>
18	<code> m.array_accesses++;</code>
19	<code> }</code>
20	
21	<code> for (int i = 1; i < 10; i++) {</code>
22	<code> count[i] += count[i - 1];</code>
23	<code> m.array_accesses += 2;</code>
24	<code> }</code>
25	
26	<code> for (int i = n - 1; i >= 0; i--) {</code>
27	<code> output[count[(data[i] / exp) % 10] - 1] = data[i];</code>
28	<code> m.array_accesses += 2;</code>
29	
30	<code> count[(data[i] / exp) % 10]--;</code>
31	<code> m.array_accesses++;</code>
32	<code> }</code>
33	
34	<code> for (int i = 0; i < n; i++) {</code>
35	<code> data[i] = output[i];</code>
36	<code> m.array_accesses++;</code>
37	<code> }</code>
38	<code>}</code>

```

39 void radix_sort(std::vector<int>& data, Metrics& m) {
40     int n = data.size();
41     if (n <= 1) return;
42     auto start = Clock::now();
43
44     int max_val = getMax(data, n, m);
45
46     for (int exp = 1; max_val / exp > 0; exp *= 10) {
47         countSort(data, n, exp, m);
48     }
49     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
50 }

```

B. Output Program

>> Quick Sort Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	676	396	0	131	0	0.003300
Quick Sort	Sorted	100	4950	5049	0	199	0	0.004000
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.003500
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.003100
Quick Sort	Many Duplicates	100	832	707	0	181	0	0.002200
Quick Sort	Random	1000	10530	6168	0	1337	0	0.033400
Quick Sort	Sorted	1000	499500	500499	0	1999	0	0.345700
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	0.198000
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.045100
Quick Sort	Many Duplicates	1000	12588	9665	0	1801	0	0.037500
Quick Sort	Random	10000	149797	77759	0	13347	0	0.388500
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	25.188100
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	17.405100
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	0.449700
Quick Sort	Many Duplicates	10000	170629	113205	0	18007	0	0.356900

Gambar 7. Output Algoritma Radix Sort

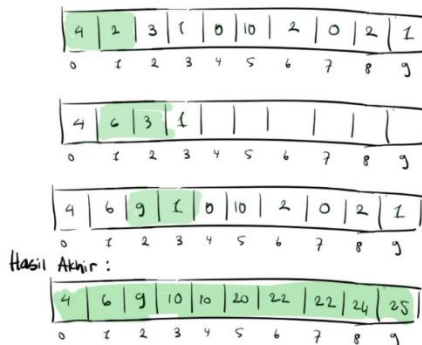
C. Pembahasan

Di dalam fungsi `getMax`, menetapkan elemen pertama sebagai nilai maksimum `mx = data[0]`. Terdapat perulangan untuk memeriksa seluruh data di dalam array. Di perulangan tersebut terdapat kondisi, jika `data[i] > mx` maka nilai `mx` akan diganti menjadi data indeks `i` yang lebih besar daripada nilai `mx` menjadi `data[i]`. Setelah itu, mengembalikan nilai `mx`.

Di dalam fungsi `countSort`, `Output(n)` berfungsi untuk menyimpan hasil urutan sementara dan array `count[10] = { 0 }` untuk menghitung berapa kali digit 0-9 muncul. Terdapat empat perulangan `for` yang berjalan secara berurutan:

1. Perulangan pertama berjalan sebanyak `i=0` sampai `i<n`, menghitung jumlah kemunculan setiap digit dari satu elemen dari data tiap iterasinya, menggunakan rumus $(data[i] / exp) \% 10$. Sebagai contoh 184 akan menghasilkan satuan 4 lalu `count[4]` bertambah satu. Proses ini terus dilakukan sampai seluruh elemen selesai diperiksa.

- Setelah seluruh digit dihitung, perulangan kedua dari $i=1$ hingga $i<10$ mengubah array count menjadi posisi kumulatif agar program tahu posisi akhir setiap digit array dengan $\text{count}[i] += \text{count}[i - 1]$.



Gambar 8. Gambaran Kumulatif

- Selanjutnya, perulangan ketiga berulang sebanyak $i = n-1$ sampai $i \geq 0$, dimana tahap ini mengecek setiap elemen dari bagian paling kanan ke kiri dan memasukkannya ke posisi array output dengan $\text{count}[(\text{data}[i] / \text{exp}) \% 10] - 1$. Setelah elemen ditempatkan, nilai count untuk digit tersebut dikurangi satu agar posisi berikutnya berpindah ke indeks sebelumnya.
- perulangan terakhir dari $i=0$ hingga $i<n$ menyalin seluruh array output dan mengembalikannya ke array data.

Dalam fungsi `radix_sort`, memanggil fungsi `getMax(data, n, m)` untuk mencari nilai tertinggi dan mengetahui digit maksimumnya. Setelah itu, program melakukan perulangan `for (int exp = 1; max_val / exp > 0; exp *= 10)` yang akan berjalan terus berdasarkan banyak digit nilai tertinggi tersebut, nilai max tersebut akan dibagi oleh `exp`, untuk perulangan pertama akan mengurutkan satuan, lalu pada perulangan kedua akan mengurutkan puluhan. `exp` akan bertambah `exp*=10` sampai syarat `max_val / exp > 0` tidak lagi terpenuhi.

Di setiap iterasinya, memanggil `countSort(data, n, exp, m)` untuk merapikan data berdasarkan digitnya. misal satuan, puluhan, ratusan atau ribuan dan seterusnya. Setelah fase perulangan berakhir, seluruh data akan terurut dari yang terkecil.

Berdasarkan hasil output, Best, Average, dan Worst Case adalah $O(n*d)$ karena cara kerjanya yang murni berupa perulangan di dalam perulangan. Dari output, terlihat kompleksitas waktu yang sama rata di ketiga kasus konsisten dan stabil

Algoritma Radix Sort termasuk Stable karena data dipindahkan secara mundur dari belakang sehingga urutan relatif asli tidak berubah. Algoritma ini tidak termasuk In-Place Sorting karena membutuhkan alokasi memori tambahan.

KESIMPULAN

Berdasarkan output hasil, berikut adalah urutan algoritma dari yang paling cepat:

1. Radix Sort
2. Quick Sort
3. Merge Sort
4. Insertion Sort
5. Selection Sort
6. Bubble Sort

Radix Sort adalah algoritma tercepat pertama dengan waktu eksekusi rata-rata sekitar 0.16 ms – 0.17 ms ketika $N = 10.000$. Waktu eksekusi percobaan semuanya di bawah 1.0 ms pada seluruh pengujian Sedangkan bubble sort paling lambat terakhir karena harus membandingkan dan menukar elemen terlebih dahulu. Ini membuat bubble sort tidak efisien untuk data besar

TAUTAN GIT

<https://github.com/DSA26-ULM/module-4-sorting-jyaritta>